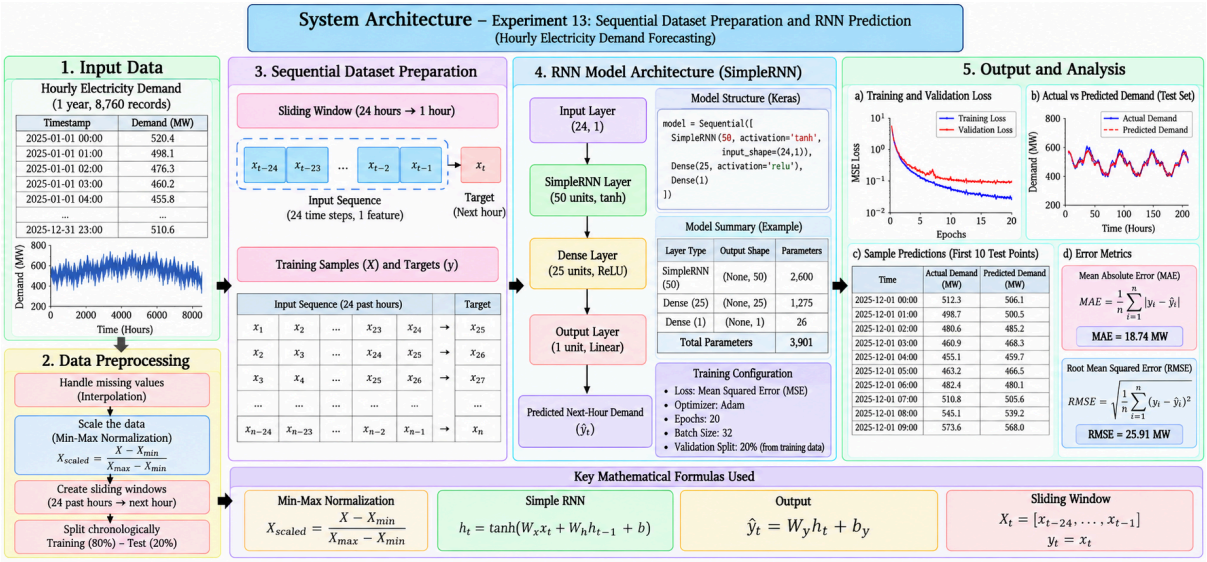
Experiment 13: Sequential Dataset Preparation and RNN Prediction

1. Aim

To develop a Simple Recurrent Neural Network (SimpleRNN) model for time-series forecasting of hourly electricity demand. To handle missing values, normalize the time-series data, convert it into supervised learning sequences using a sliding-window technique, split the data chronologically into training and testing sets, and predict future electricity demand. The model performance is evaluated using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) with TensorFlow, Keras, NumPy, Pandas, and Matplotlib.



2. Mathematical Formulas

1. Min-Max Normalization

$$\boxed{X_{\text{scaled}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}}$$

2. Sliding Window

For a window size of 24:

$$\boxed{X_t = [x_{t-24}, x_{t-23}, \ldots, x_{t-1}]}$$

$$\boxed{Y_t = x_t}$$

The previous 24 hours are used to predict the next hour.

3. Simple RNN Calculation

$$\boxed{h_t = \tanh(W_x x_t + W_h h_{t-1} + b)}$$

where h_t represents the hidden state at time t.

4. Output Prediction

$$\boxed{\hat{y}_t = W_y h_t + b_y}$$

5. Mean Absolute Error (MAE)

$$\boxed{MAE = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|}$$

6. Root Mean Squared Error (RMSE)

$$\boxed{RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}}$$

7. Chronological Data Splitting

$$\boxed{\text{Training Data} = 80\%}$$

$$\boxed{\text{Testing Data} = 20\%}$$

```python
# Experiment 13: Sequential Dataset Preparation and RNN Prediction

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

np.random.seed(42)
tf.random.set_seed(42)

# ========================================
# 1. GENERATE HOURLY ELECTRICITY DATA
# ========================================

hours = 365 * 24

dates = pd.date_range(
    start="2025-01-01",
    periods=hours,
    freq="h"
)

t = np.arange(hours)

# Simulate hourly electricity demand
demand = (
    500
    + 80 * np.sin(2 * np.pi * t / 24)
    + 120 * np.sin(2 * np.pi * t / (24 * 365))
    + 40 * np.sin(2 * np.pi * t / 168)
    + np.random.normal(0, 20, hours)
)

df = pd.DataFrame({
    "Date": dates,
    "Demand": demand
})

# Introduce missing values for demonstration
missing_indices = np.random.choice(
    hours, size=30, replace=False
)
df.loc[missing_indices, "Demand"] = np.nan

print("Original Dataset:")
print(df.head())

# ========================================
# 2. HANDLE MISSING VALUES
# ========================================

df["Demand"] = df["Demand"].interpolate(
    method="linear"
)

print("\nMissing Values After Interpolation:")
print(df.isnull().sum())

# ========================================
# 3. SCALE THE TIME SERIES
# ========================================
```

```
values = df["Demand"].values.reshape(-1, 1)

# Chronological split
split_index = int(len(values) * 0.80)

train_values = values[:split_index]

scaler = MinMaxScaler()

# Fit scaler only on training data
scaler.fit(train_values)

scaled_values = scaler.transform(values)

print("\nScaled Data Sample:")
print(scaled_values[:5])

# ========================================
# 4. CREATE SLIDING WINDOWS
# ========================================

window_size = 24

X, y, target_indices = [], [], []

for i in range(window_size, len(scaled_values)):
    X.append(scaled_values[i-window_size:i])
    y.append(scaled_values[i])
    target_indices.append(i)

X = np.array(X)
y = np.array(y)
target_indices = np.array(target_indices)

# Split according to target timestamps
train_mask = target_indices < split_index
test_mask = target_indices >= split_index

X_train, y_train = X[train_mask], y[train_mask]
X_test, y_test = X[test_mask], y[test_mask]

print("\nSequence Shapes:")
print("X_train:", X_train.shape)
print("y_train:", y_train.shape)
print("X_test:", X_test.shape)
print("y_test:", y_test.shape)

# ========================================
# 5. BUILD SIMPLE RNN MODEL
# ========================================

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(24, 1)),

    tf.keras.layers.SimpleRNN(
        50, activation="tanh"
    ),

    tf.keras.layers.Dense(25, activation="relu"),

    tf.keras.layers.Dense(1)
])
```

```
model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)

model.summary()

# ========================================
# 6. TRAIN THE RNN MODEL
# ========================================

history = model.fit(
    X_train, y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.2,
    shuffle=False,
    verbose=1
)

# ========================================
# 7. PLOT TRAINING AND VALIDATION LOSS
# ========================================

plt.figure(figsize=(8, 5))

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title("RNN Training and Validation Loss")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# ========================================
# 8. PREDICT TEST DATA
# ========================================

predictions_scaled = model.predict(
    X_test, verbose=0
)

# Convert predictions back to original scale
y_actual = scaler.inverse_transform(y_test)
y_predicted = scaler.inverse_transform(predictions_scaled)

# ========================================
# 9. CALCULATE MAE AND RMSE
# ========================================

mae = mean_absolute_error(
    y_actual, y_predicted
)

rmse = np.sqrt(
    mean_squared_error(y_actual, y_predicted)
)

print("\nMODEL EVALUATION")
print("MAE :", round(mae, 4))
print("RMSE:", round(rmse, 4))
```

```
# ========================================
# 10. PLOT ACTUAL VS PREDICTED DEMAND
# ========================================

plt.figure(figsize=(12, 5))

plt.plot(
    y_actual[:200],
    label="Actual Demand"
)

plt.plot(
    y_predicted[:200],
    label="Predicted Demand"
)

plt.xlabel("Time (Hours)")
plt.ylabel("Electricity Demand")
plt.title("Actual vs Predicted Electricity Demand")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# ========================================
# 11. DISPLAY FINAL PREDICTION TABLE
# ========================================

comparison = pd.DataFrame({
    "Actual Demand": y_actual.ravel()[:10],
    "Predicted Demand": y_predicted.ravel()[:10]
})

print("\nActual vs Predicted Values:")
print(comparison.round(2))

print("\nExperiment Completed Successfully!")
```

Original Dataset:

	Date	Demand
0	2025-01-01 00:00:00	509.934283
1	2025-01-01 01:00:00	519.521956
2	2025-01-01 02:00:00	556.115117
3	2025-01-01 03:00:00	591.765932
4	2025-01-01 04:00:00	570.904939

Missing Values After Interpolation:

```
Date      0
Demand    0
dtype: int64
```

Scaled Data Sample:

```
[[0.51078491]
 [0.52790345]
 [0.59323956]
 [0.65689314]
```

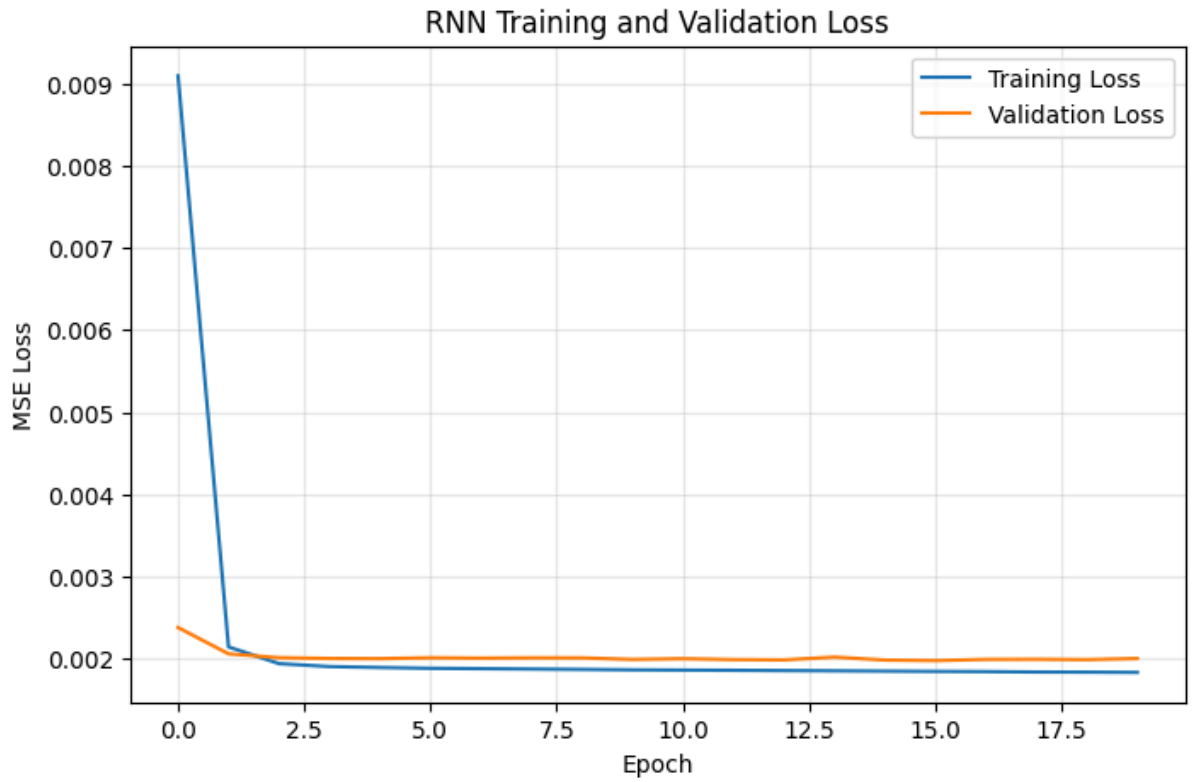
[0.61964639]]

Sequence Shapes:
X_train: (6984, 24, 1)
y_train: (6984, 1)
X_test: (1752, 24, 1)
y_test: (1752, 1)

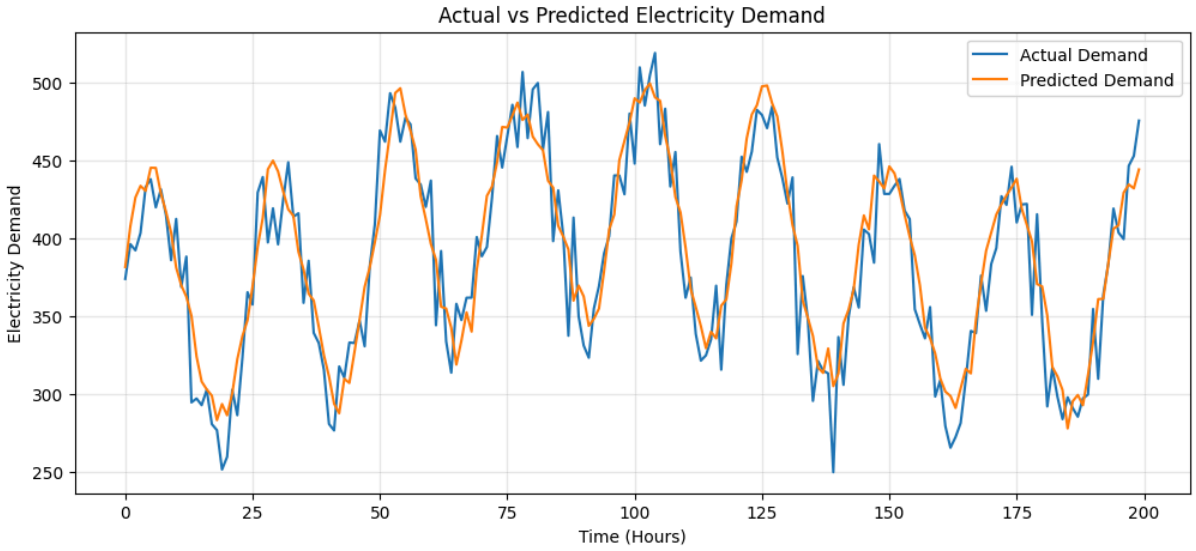
```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 50)	2,600
dense (Dense)	(None, 25)	1,275
dense_1 (Dense)	(None, 1)	26

```
 Total params: 3,901 (15.24 KB)
 Trainable params: 3,901 (15.24 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.0091 - mae: 0.0632 - val_loss: 0.0024 - val_mae: 0.0394
Epoch 2/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - loss: 0.0021 - mae: 0.0373 - val_loss: 0.0021 - val_mae: 0.0367
Epoch 3/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0019 - mae: 0.0353 - val_loss: 0.0020 - val_mae: 0.0362
Epoch 4/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0019 - mae: 0.0350 - val_loss: 0.0020 - val_mae: 0.0361
Epoch 5/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0019 - mae: 0.0349 - val_loss: 0.0020 - val_mae: 0.0360
Epoch 6/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0019 - mae: 0.0348 - val_loss: 0.0020 - val_mae: 0.0361
Epoch 7/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0019 - mae: 0.0347 - val_loss: 0.0020 - val_mae: 0.0361
Epoch 8/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0019 - mae: 0.0347 - val_loss: 0.0020 - val_mae: 0.0361
Epoch 9/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0019 - mae: 0.0346 - val_loss: 0.0020 - val_mae: 0.0361
Epoch 10/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0019 - mae: 0.0346 - val_loss: 0.0020 - val_mae: 0.0359
Epoch 11/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0019 - mae: 0.0345 - val_loss: 0.0020 - val_mae: 0.0361
Epoch 12/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0019 - mae: 0.0345 - val_loss: 0.0020 - val_mae: 0.0360
Epoch 13/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - loss: 0.0019 - mae: 0.0345 - val_loss: 0.0020 - val_mae: 0.0359
Epoch 14/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0019 - mae: 0.0344 - val_loss: 0.0020 - val_mae: 0.0362
Epoch 15/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0018 - mae: 0.0344 - val_loss: 0.0020 - val_mae: 0.0359
Epoch 16/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0018 - mae: 0.0343 - val_loss: 0.0020 - val_mae: 0.0358
Epoch 17/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0018 - mae: 0.0343 - val_loss: 0.0020 - val_mae: 0.0360
Epoch 18/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0018 - mae: 0.0343 - val_loss: 0.0020 - val_mae: 0.0360
Epoch 19/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.0018 - mae: 0.0343 - val_loss: 0.0020 - val_mae: 0.0359
Epoch 20/20
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0018 - mae: 0.0342 - val_loss: 0.0020 - val_mae: 0.0361
```



MODEL EVALUATION
MAE : 19.5774
RMSE: 24.4022



Actual vs Predicted Values:

	Actual Demand	Predicted Demand
0	373.81	381.459991
1	396.18	407.690002
2	392.20	426.220001

3	403.42	433.660004
4	432.83	430.459991
5	437.95	445.290009
6	419.85	445.220001
7	431.46	428.809998
8	414.78	417.010010
9	385.86	403.809998

5. RESULT

A Simple Recurrent Neural Network (SimpleRNN) was implemented using TensorFlow and Keras for hourly electricity-demand forecasting. The time-series dataset was loaded using Pandas, missing values were handled through interpolation, and the data was normalized using Min-Max scaling. Sliding windows of 24 previous hours were created to predict the next hour. The model was trained using chronological training and testing data. Actual and predicted electricity demand were visualized, and model performance was evaluated using MAE and RMSE. Thus, the experiment demonstrates how RNNs learn sequential patterns and forecast future values from historical time-series data.